

1. What are the differences between Native, Hybrid, and Cross-platform mobile application development, and when is each approach commonly used?

- **Native Development** means creating an app specifically for one operating system, such as Android or iOS. Developers use the official programming languages and tools (like Kotlin for Android and Swift for iOS). Native apps usually have the best performance, smoother user experience, and full access to device features. This approach is commonly used for apps that require high speed, advanced graphics, or top-level performance, such as games, banking apps, and camera applications.
- **Hybrid Development** combines web technologies like HTML, CSS, and JavaScript and wraps them inside a mobile app. A single codebase can run on both Android and iOS, making development faster and more affordable. Hybrid apps are commonly used for simple business apps, information apps, or projects with limited budgets and shorter development timelines.
- **Cross-platform Development** allows developers to write one codebase that works on multiple platforms while still providing a near-native experience. Frameworks like Flutter and React Native are popular examples. This approach is commonly used for startups, business applications, and projects that need to support both Android and iOS without building separate apps.

2. What is the Flutter Ecosystem, and how does it help developers create modern mobile applications?

- The **Flutter Ecosystem** is the collection of tools, libraries, packages, and resources that support Flutter development. It includes the Flutter framework, the Dart programming language, ready-made widgets, third-party packages, development tools, and a large developer community.

This ecosystem helps developers by making app development faster and easier. Instead of building every feature from scratch, developers can use existing packages for things like authentication, databases, maps, and notifications. Flutter also provides customizable widgets that make it easy to create attractive and responsive user interfaces. Because Flutter uses one codebase for multiple platforms, developers save time while still creating high-quality applications.

3. What is the Software Development Life Cycle (SDLC), and why is it important in building successful applications?

- The **Software Development Life Cycle (SDLC)** is a step-by-step process used to plan, create, test, and maintain software applications. Its main stages include planning, gathering requirements, designing, developing, testing, deploying, and maintaining the application.

The SDLC is important because it provides a clear structure for the entire development process. It helps developers stay organized, reduces errors, improves communication among team members, and ensures that the final product meets the needs of users. Following the SDLC also makes projects easier to manage, complete on time, and maintain after they are released.

4. How do Flutter development practices and the SDLC work together in creating real-world mobile applications?

- Flutter and the SDLC complement each other throughout the development process. During the planning and design stages, developers decide how the app should work and how Flutter's widgets and features will be used. During development, Flutter allows developers to quickly build and test features using tools like Hot Reload, which speeds up coding and debugging.

As the project moves into testing, Flutter supports testing tools that help ensure the app works properly on different devices. Finally, during deployment and maintenance, developers can easily update the application, fix bugs, and add new features while following the SDLC process. Together, Flutter and the SDLC help teams build reliable, efficient, and user-friendly mobile applications.

5. Why should developers understand these concepts before starting an application project?

- Developers should understand these concepts because they help them choose the right approach for building an application. Knowing the differences between native, hybrid, and cross-platform development allows them to select the best technology based on the project's goals, budget, and target users.

Understanding the Flutter ecosystem helps developers work more efficiently by using the right tools and resources. Learning the SDLC teaches them how to plan, develop, test, and maintain applications in an organized way. When developers understand all of these concepts before starting a project, they can avoid common mistakes, make better decisions, work more effectively as a team, and create applications that are reliable, user-friendly, and successful.